

VisionQuest – Projektdokumentation

Projekt: VisionQuest – AR-basierte Objekterkennungs-Quest-App

Version: 1.0.0

Stand: März 2026

Team: WatzingerM21052

Inhaltsverzeichnis

1. [Projektstart – Schnellübersicht](#)
2. [Installation & Inbetriebnahme](#)
 - 2.1 [Systemvoraussetzungen](#)
 - 2.2 [Backend starten](#)
 - 2.3 [Frontend starten](#)
 - 2.4 [Python / YOLO \(optional\)](#)
 - 2.5 [Verbindung prüfen](#)
 - 2.6 [Häufige Probleme & Lösungen](#)
3. [Protokoll-Sammlung](#)
4. [Meilenstein-Plan vs. tatsächlicher Fortschritt](#)
5. [App-Bedienung – Schritt-für-Schritt](#)
 - 5.1 [Registrierung](#)
 - 5.2 [Login](#)
 - 5.3 [Home-Screen \(Dashboard\)](#)
 - 5.4 [Scanner – Objekt erkennen](#)
 - 5.5 [Reward-Screen](#)
 - 5.6 [Quest-Log](#)
 - 5.7 [Einstellungen](#)
 - 5.8 [Logout](#)

1. Projektstart – Schnellübersicht

VisionQuest ist eine Flutter-App, die mit einer Kamera reale Objekte erkennt und dem Spieler dafür XP, Level-Aufstiege und Streak-Punkte vergibt. Die Erkennung läuft wahlweise über **YOLO v11** (Python) oder **COCO-SSD** (TensorFlow.js) im Express-Backend.

```
Frontend (Flutter/Dart) ↔ Backend (Node.js/Express) ↔ SQLite-DB
                        ↓
                    YOLO v11 (Python) / COCO-SSD (TF.js)
```

Starten in zwei Schritten:

```
# Terminal 1 - Backend
cd Project/backend && npm install && npm start

# Terminal 2 - Frontend
cd Project/frontend && flutter pub get && flutter run -d chrome
```

2. Installation & Inbetriebnahme

2.1 Systemvoraussetzungen

Komponente	Mindestversion	Hinweis
Flutter SDK	3.10.3	<code>flutter --version</code>
Dart	3.0	im Flutter SDK enthalten
Node.js	16 LTS	<code>node --version</code>
npm	8	<code>npm --version</code>
Python	3.13	nur für YOLO; optional
Ultralytics	8.4.19	<code>pip install ultralytics==8.4.19</code>
Android SDK/Xcode	aktuell	nur für native Builds
Betriebssystem	Win 10/macOS 12/Ubuntu 20	beliebig

2.2 Backend starten

```
# 1. In den Backend-Ordner wechseln
cd Project/backend

# 2. Node-Abhängigkeiten installieren (einmalig)
npm install

# 3. (Optional) .env anlegen - falls nicht vorhanden
#   Folgende Werte sind die Standardwerte und können weggelassen werden:
#   PORT=5000
#   JWT_SECRET=visionquest_secret
#   VISION_YOLO_MODEL=yolo11n.pt
#   VISION_YOLO_CONF=0.20
#   VISION_YOLO_IMGSZ=640

# 4. Server starten
npm start
# Alternativ im Entwicklermodus:
# npm run dev

# ✅ Erwartete Ausgabe:
# Server running on port 5000
# Database initialized successfully
```

Der Backend-Server ist danach erreichbar unter:

<http://localhost:5000>

Backend-Tests ausführen:

```
npm test           # Unit-Tests mit Jest
npm run test:watch # Tests im Watch-Modus
```

2.3 Frontend starten

```
# 1. In den Frontend-Ordner wechseln
cd Project/frontend

# 2. Flutter-Abhängigkeiten installieren (einmalig)
flutter pub get

# 3. Code-Qualität prüfen
flutter analyze
# ✅ Erwartete Ausgabe: "No issues found!"

# 4. App starten - Plattform wählen:

# Web (empfohlen für schnelles Testen)
flutter run -d chrome

# Android-Emulator (muss gestartet sein)
flutter run -d emulator-5554

# iOS-Simulator (nur macOS)
flutter run -d iphone

# Windows Desktop
flutter run -d windows

# Alle verfügbaren Geräte anzeigen
flutter devices
```

Hinweis: Bei einem **Android-Emulator** oder **physischen Gerät** muss die Backend-IP angepasst werden.

In frontend/lib/main.dart den baseUrl-Wert ändern:

- Android Emulator: `http://10.0.2.2:5000/api`
- Physisches Gerät: `http://<DEINE-LAN-IP>:5000/api`
- Web / Desktop: `http://localhost:5000/api`

Release-Build erstellen:

```
# Android APK
flutter build apk --release

# Web
flutter build web --release --web-renderer canvaskit

# Windows
flutter build windows --release
```

2.4 Python / YOLO (optional)

YOLO v11 erhöht die Erkennungsgenauigkeit erheblich. Das Backend fällt automatisch auf COCO-SSD zurück, wenn Python nicht gefunden wird.

```
# Python-Paket installieren
pip install ultralytics==8.4.19

# Installation prüfen
python -c "from ultralytics import YOLO; print('YOLO OK!)"

# Modell vorab herunterladen (optional; ~6 MB)
# Sonst erfolgt der Download automatisch beim ersten Scan
python -c "from ultralytics import YOLO; YOLO('yolo11n.pt!)"
```

Umgebungsvariablen für das Backend (.env):

```
VISION_YOLO_MODEL=yolo11n.pt    # Modell-Datei
VISION_YOLO_CONF=0.20           # Konfidenz-Schwellwert
VISION_YOLO_IMGSZ=640           # Eingangsbildgröße
```

2.5 Verbindung prüfen

Beim App-Start prüft das Frontend automatisch die Backend-Verbindung und zeigt einen farbigen Status:

Symbol	Bedeutung
✓ Grün	Backend erreichbar – alles OK
✗ Rot	Backend nicht erreichbar

Mit dem Button „**Verbindung testen**“ kann die Prüfung jederzeit wiederholt werden.

2.6 Häufige Probleme & Lösungen

Problem	Ursache	Lösung
<code>npm: command not found</code>	Node.js nicht installiert	Node.js von nodejs.org installieren
Port 5000 belegt	Anderer Prozess	<code>.env</code> : <code>PORT=3001</code> setzen und <code>baseUrl</code> im Frontend anpassen
<code>flutter: command not found</code>	Flutter nicht im PATH	Flutter SDK zur PATH-Variable hinzufügen
<code>flutter pub get</code> schlägt fehl	kein Internet / <code>pub.dev</code> nicht erreichbar	VPN prüfen oder <code>flutter pub cache repair</code>
App baut, zeigt aber leeren Screen	Backend läuft nicht	<code>npm start</code> im Backend-Ordner ausführen
YOLO schlägt fehl	Python nicht in PATH	<code>python --version</code> testen; Fallback auf COCO-SSD aktiv
Kamera funktioniert nicht (Web)	Browser-Berechtigung fehlt	Browser → Kamera-Erlaubnis erteilen
Android Emulator erreicht Backend nicht	IP-Adresse falsch	<code>baseUrl</code> auf <code>http://10.0.2.2:5000/api</code> setzen
<code>flutter analyze</code> zeigt Fehler	Code-Probleme	<code>flutter clean</code> && <code>flutter pub get</code> ausführen
Lottie-Animation lädt nicht	Kein Internet / CDN-Fehler	Fallback auf Material-Icons aktiv – kein Handlungsbedarf

3. Protokoll-Sammlung

Wochenprotokoll – 30.01.2026

Was wurde gemacht?

Erstellung des Repositorys und der Ordnerstruktur sowie Start des Projektkonzepts inklusive erster Mockups.

Zeitplan:

Gut im Zeitplan, da ein Großteil des Konzepts bereits vor den Semesterferien erstellt wurde.

Wochenprotokoll – 20.02.2026 (Meilenstein M1)

Was wurde gemacht?

Initialisierung der Flutter-Projektstruktur, Konfiguration des Express.js Backends mit CORS sowie Definition des SQLite-Schemas mit vier Tabellen (users, quests, user_stats, achievements).

Zeitplan:

Im Plan; alle initialen Setup-Tasks und die Basis-Infrastruktur wurden abgeschlossen.

KI-Einsatz:

Nutzung von GitHub Copilot (GPT-4) für Strukturvorschläge, Try-Catch-Patterns bei API-Antworten und die Erstellung eines `/api/health` Test-Endpunkts.

Wochenprotokoll – 27.02.2026 (Meilenstein M2)

Was wurde gemacht?

Implementierung von CRUD-Operationen für User und Quests, Definition von 13 REST-API Endpunkten sowie Integration von JWT-Authentication und Passwort-Hashing (bcryptjs).

Zeitplan:

Gut im Zeitplan; alle für den 02.03. geplanten Tasks wurden vorzeitig abgeschlossen.

KI-Einsatz:

GitHub Copilot half bei der Standardisierung des Error-Handlings, der Fehlerbehebung bei der JWT-

Validierung und der Logik zur Vermeidung doppelter Benutzerregistrierungen.

Wochenprotokoll – 06.03.2026 (Meilenstein M5)

Was wurde gemacht?

Integration des camera-Plugins in Flutter, Implementierung des Live-Preview-Screens und des Bild-Uploads an den Backend-Endpunkt `/api/detect`. Das Backend wurde um einen asynchronen Python-Subprozess zur Bildverarbeitung erweitert.

Zeitplan:

Im Plan; die Kernfunktionalität für Computer Vision ist einsatzbereit.

KI-Einsatz:

Unterstützung durch Copilot bei Kamera-Berechtigungen (Android/iOS), der korrekten Darstellung der Preview via AspectRatio und der Integration des Python-Scripts in Node.js mittels `child_process.spawn`.

Wochenprotokoll – 13.03.2026 (Meilenstein M6+M7+M8)

Was wurde gemacht?

Vollständige Implementierung aller 6 Screens, zentralisiertes Routing, Integration von Riverpod für das State Management sowie Einbau von 5 Themes und der Quest-Logik (XP, Level, Streak). Zudem wurden Lottie/Rive Animationen hinzugefügt.

Zeitplan:

Im Plan; alle Ziele für UI, Navigation, State und Themes wurden termingerecht zum Projektabschluss erreicht.

KI-Einsatz:

Einsatz von GitHub Copilot (GPT-5.3-Codex) für das Named-Routing, zentrale Provider-Strukturen, Berechnungsregeln für die Quest-Logik und ein sicheres Code-Cleanup.

4. Meilenstein-Plan vs. tatsächlicher Fortschritt

Meilenstein	Geplante Funktionalität	Deadline (Soll)	Status (Ist)
M1	Projektplanung & Initialisierung (Git, Konzept, Mockups)	10.02.2026	✅ Abgeschlossen (30.01.)
M2	Projekt-Init (Flutter Struktur, Express Boilerplate)	27.02.2026	✅ Abgeschlossen (20.02.)
M3	Datenbank & API (SQLite Schema, CRUD, Endpunkte)	02.03.2026	✅ Abgeschlossen (27.02.)
M4	Authentication (Login/Register UI, JWT, Session)	04.03.2026	✅ Abgeschlossen (27.02.)
M5	Scanner & Computer Vision (Kamera, Upload, Detection)	06.03.2026	✅ Abgeschlossen (06.03.)
M6	UI & Navigation (Alle 6 Screens, Responsive Layout)	09.03.2026	✅ Abgeschlossen (13.03.)
M7	State & Theme (Riverpod, 5 Themes, Quest-Logik)	11.03.2026	✅ Abgeschlossen (13.03.)
M8	Finale Abgabe & Dokumentation	13.03.2026	✅ Termingerecht

Fazit zum Fortschritt:

Das Projekt konnte exakt im vorgegebenen Zeitrahmen abgeschlossen werden. Während die Backend-Entwicklung und Datenbankstruktur (M3, M4) dem Plan voraus waren, lag der Fokus in der letzten Woche auf der Finalisierung der UI-Komplexität und dem globalen State Management (M6, M7). Alle im ursprünglichen Konzept definierten Anforderungen – von der JWT-Authentifizierung bis zum variablen Theme-System – wurden vollständig umgesetzt.

5. App-Bedienung – Schritt-für-Schritt

5.1 Registrierung

Die App startet direkt auf dem **Login-Screen**. Neue Nutzer wählen „Registrieren“.

Schritte:

1. App öffnen → Login-Screen erscheint
2. Auf „**Registrieren**“ tippen (Link am unteren Bildschirmrand)
3. Felder ausfüllen:
 - Benutzername – frei wählbar, eindeutig
 - E-Mail – gültige E-Mail-Adresse
 - Passwort – mindestens 6 Zeichen
4. Auf „**Registrieren**“ drücken
5. Bei Erfolg → automatische Weiterleitung zum **Home-Screen**

DEBUG



Konto erstellen

Tritt VisionQuest bei und finde geheime
Objekte



Registrieren

oder

Bereits ein Konto? [Zum Login](#)

Hinweis: Bei Fehlern (doppelte E-Mail, kurzes Passwort) erscheint eine Fehlermeldung direkt unter dem betroffenen Feld.

5.2 Login

Schritte:

1. App öffnen → Login-Screen erscheint
2. E-Mail und Passwort eingeben
3. Auf „**Anmelden**“ tippen
4. Bei Erfolg → Weiterleitung zum **Home-Screen**

DEBUG



Willkommen zurück

Melde dich an um dein Abenteuer
fortzusetzen



E-Mail



Passwort



Anmelden

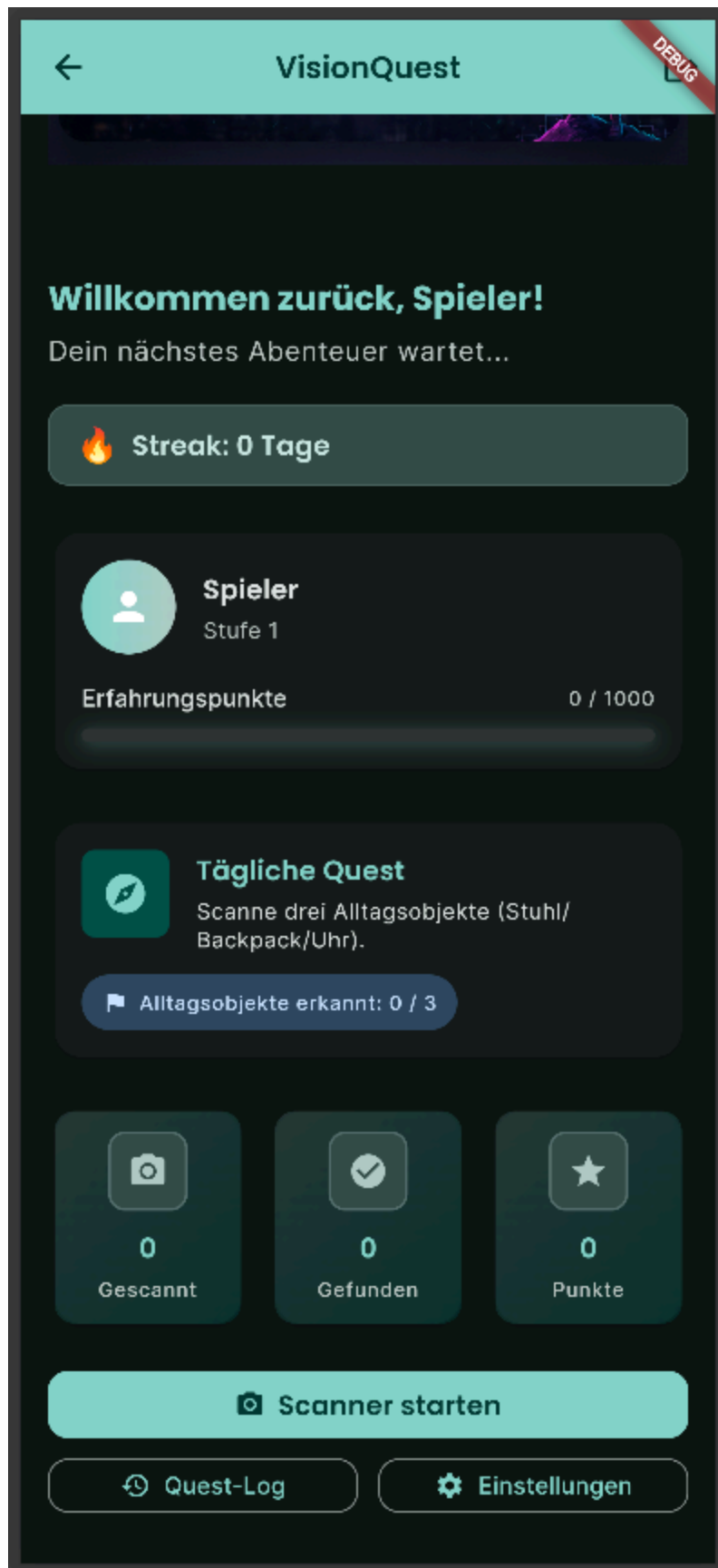
oder

Noch kein Konto? [Jetzt registrieren](#)

Tipp: Das Auth-Token wird sicher gespeichert. Bei erneutem Öffnen der App muss nicht erneut eingeloggt werden, solange die Session aktiv ist.

5.3 Home-Screen (Dashboard)

Der Home-Screen ist die zentrale Übersicht. Er zeigt den aktuellen Spielstand.



Element	Beschreibung
Begrüßung	Personalisiert mit eingeloggtem Benutzernamen
Level-Karte	Zeigt aktuelles Level, XP-Fortschrittsbalken (mit Glow-Effekt)
Streak-Anzeige	Animiertes 🔥 -Icon mit Pulse-Effekt; zeigt konsekutive Tage
Tagesquest	Heute aktive Quest mit Fortschrittsanzeige; wechselt täglich
Statistiken	Gescannt = alle Scans, Gefunden = erfolgreiche Erkennungen
Buttons	Navigation zu Scanner, Quest-Log und Einstellungen

Daily Quest: Die aktive Quest wechselt täglich und rotiert durch 6 Quests (Handy, Personen, Tasse, Buch, Alltagsobjekte, Technik). Der Fortschritt wird automatisch aus dem Quest-Log berechnet.

5.4 Scanner – Objekt erkennen

Der Scanner-Screen zeigt das Live-Kamerabild mit einem Fokus-Kreis.

Schritte:

1. Auf „**Jetzt scannen**“ tippen
2. Kameraberechtigung bestätigen (beim ersten Mal)
3. Objekt **in den Fokus-Kreis** richten
4. Auf „**Scannen**“ tippen
5. Kurze Analyse-Phase (Lottie-Spinner sichtbar)
6. Weiterleitung zum **Reward-Screen** bei Erkennung

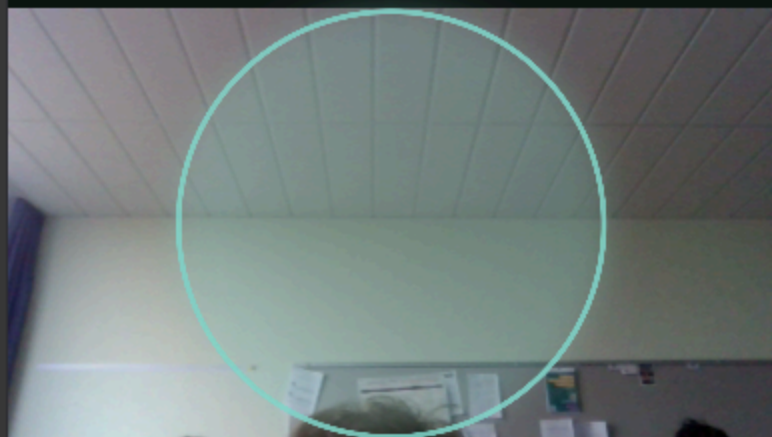


Scanner



Bereit

DEBUG



Scan starten



YOLO · Balanced



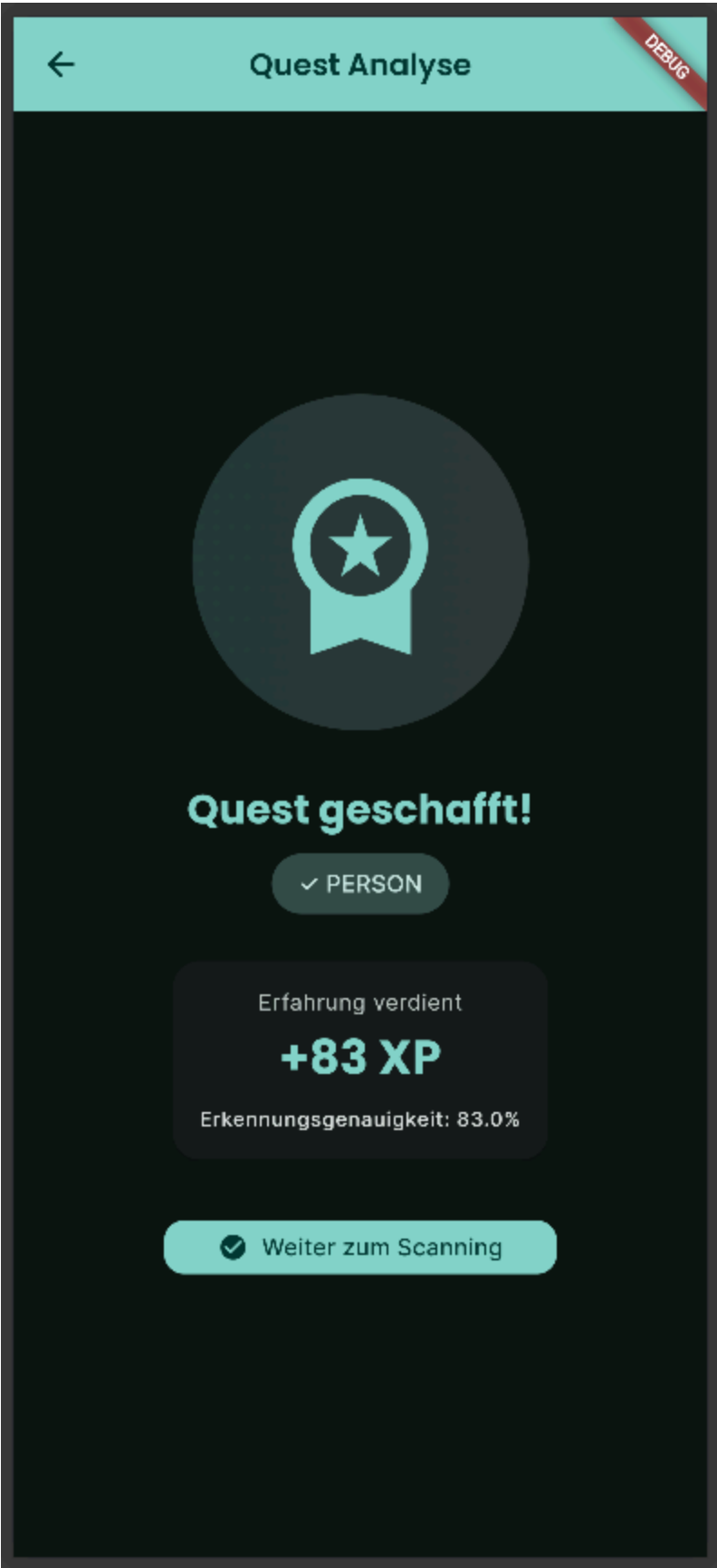
Modus	Beschreibung	Empfohlen für
Strict	Nur Objekte komplett im Kreis werden erkannt	Klare, freistehende Objekte
Balanced	Auch teilweise außerhalb des Kreises sichtbare Objekte	Große Objekte, schwierige Umgebungen
YOLO	Höhere Genauigkeit via Python/Ultralytics	Standardmodus
COCO-SSD	Schneller, kein Python nötig	Wenn YOLO nicht verfügbar

Tipp: Den Fokus-Modus und das Erkennungsmodell kann man in den **Einstellungen** wechseln. Der aktive Modus wird als kleiner Chip am unteren Rand des Scanners angezeigt.

Kein Objekt erkannt? Stelle sicher, dass das Objekt deutlich im Kreis sichtbar ist und ausreichend beleuchtet. Wechsle ggf. auf „Balanced“-Modus.

5.5 Reward-Screen

Nach einer erfolgreichen Erkennung erscheint der Reward-Screen.



Element	Beschreibung
Animation	Lottie-Feieranimation (fällt auf Material-Icon zurück, falls offline)

Element	Beschreibung
Erkanntes Objekt	Label des erkannten Objekts (englisch, aus COCO/YOLO Klassenliste)
Konfidenz	Erkennungssicherheit in Prozent
XP-Vergabe	Formel: $10 + (\text{Konfidenz} \times 90) \rightarrow 10\text{--}100 \text{ XP}$
Weiter-Button	Zurück zum Home-Screen; State (XP, Level, Streak) wird aktualisiert

XP-Beispiele:

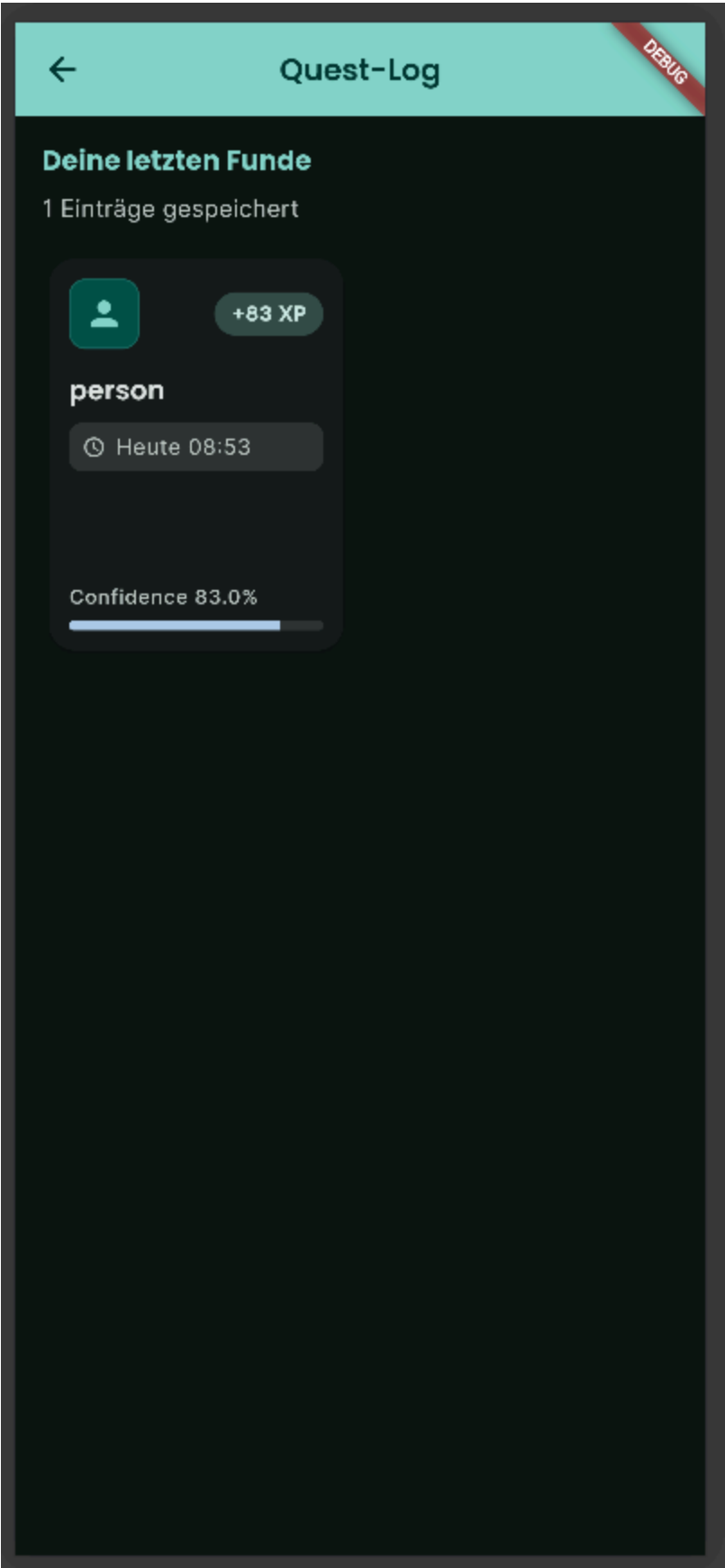
- 50 % Konfidenz $\rightarrow$ 55 XP
- 80 % Konfidenz $\rightarrow$ 82 XP
- 100 % Konfidenz $\rightarrow$ 100 XP

5.6 Quest-Log

Der Quest-Log zeigt die vollständige Historie aller Scan-Ergebnisse.

Schritte:

1. Home-Screen $\rightarrow$ „**Quest-Log**“ tippen
2. Grid-Ansicht mit allen bisherigen Scans



Element	Beschreibung
Kachel	Zeigt Objektname, Konfidenz-Balken, XP und Timestamp

Element	Beschreibung
Konfidenz-Balken	Visuell: leer (0 %) bis voll (100 %)
Grid-Layout	2 Spalten Mobile, 3+ Spalten Tablet/Desktop
Leer-State	Freundliche Nachricht, wenn noch keine Scans vorhanden

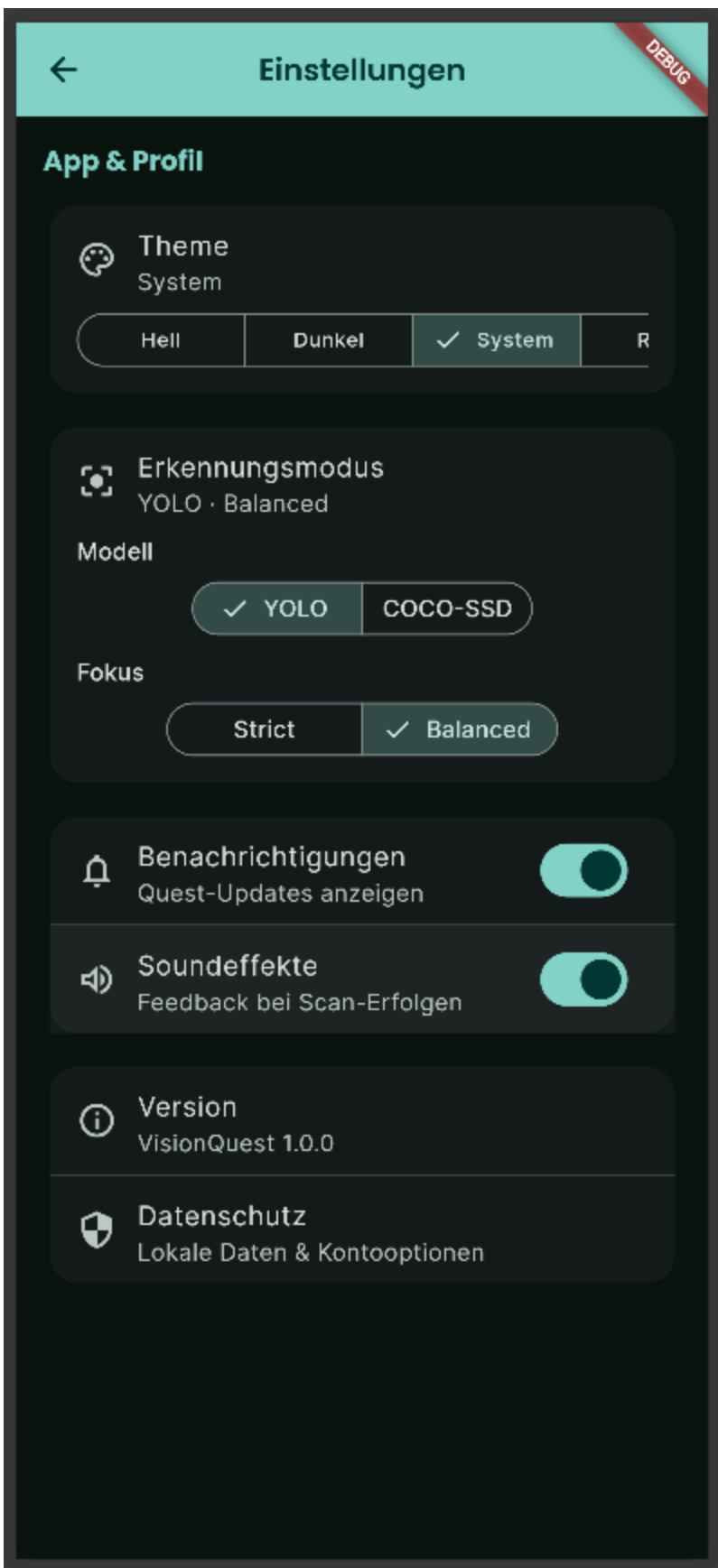
Maximale Einträge: Der Log speichert bis zu 200 Einträge; ältere Einträge werden automatisch entfernt.

5.7 Einstellungen

Die Einstellungen ermöglichen die Anpassung von Theme, Erkennungsmodell und Fokus-Modus.

Schritte:

1. Home-Screen → „**Einstellungen**“ tippen



Themes im Überblick:

Theme	Stil	Primärfarbe
Light	Hell, klassisch	Lila #5D4E8C
Dark	Dunkel, klassisch	Lila #5D4E8C
System	Folgt OS-Einstellung	Lila #5D4E8C
RetroArcade	Neon-Arcade-Look	Cyan #00C2FF
AdventureMap	Warm, rustikal	Braun #8B5E3C

Hinweis: Das Theme gilt sofort und erfordert keinen App-Neustart.

Erkennungsmodell-Optionen:

Option	Beschreibung
YOLO	YOLO v11 (Python), höhere Genauigkeit, benötigt Python
COCO-SSD	TensorFlow.js im Backend, schneller, kein Python nötig

Fokus-Optionen:

Option	Kontext-Faktor	Empfehlung
Strict	1,35×	Objekte klar sichtbar, freistehend
Balanced	1,85×	Objekte teilweise verdeckt oder am Rand

5.8 Logout

1. Home-Screen → „**Einstellungen**“ tippen
2. Am Ende der Seite „**Abmelden**“ tippen
3. Bestätigung → App leitet zum **Login-Screen** weiter
4. Auth-Token und Session-Daten werden gelöscht

Hinweis: Nach dem Logout ist der gespeicherte Auth-Token ungültig. Beim nächsten Start der App erscheint wieder der Login-Screen.

Anhang – Technologieübersicht

Tech Stack

Bereich	Technologie	Version
Frontend	Flutter	3.10.3
Sprache Frontend	Dart	3.0+
State Management	Riverpod (StateNotifier)	2.6.1
Design System	Material You (useMaterial3)	–
Animationen	Lottie + Flutter Built-in	3.1.2
Kamera	camera	0.11.0+2
Secure Storage	flutter_secure_storage	9.2.4
HTTP	http	1.6.0
Backend Framework	Express.js	5.2.1
Laufzeit Backend	Node.js	16+
Datenbank	SQLite3	5.1.7
Auth	JWT (jsonwebtoken)	9.0.3
Passwort-Hashing	bcryptjs	3.0.3
Bildverarbeitung	Jimp	0.22.10
ML Modell 1	YOLO v11 (Ultralytics)	8.4.19
ML Modell 2	COCO-SSD (TensorFlow.js)	2.2.3
Tests (Backend)	Jest + Supertest	29.7.0